Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービス

表紙

項目	内容
文書名	基本設計書
システム名	
機能ID	COM-146
画面名	Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービス
会社名	
作成日	

概要

概要説明

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスの入力・表示・要求処理を行う。

情報	内容
対象種別	common

情報	内容
開始契機	呼出元からの要求
処理対象	DB2.Ｕ職員名簿表、N2_SIGHT.Ｂ顧客基本表、受給有無テーブル、年金1_LK、年金種類テーブル、年金__削除明細、店名テーブル、性別テーブル、裁定請求テーブル

項目一覧

No	グループ	項目名	種類	入出力	DB連携項目	候補取得元	必須	形式・桁	説明

項目定義数：0項目。表示順と反復行の定義数を区別する。

イベント仕様

ReferenceDataService

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「ReferenceDataService」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連する処理へ制御を渡す。詳細な実行経路は原本対応を参照する。

getInstance

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「getInstance」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

searchBranches

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「searchBranches」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理12：詳細仕様参照）。
2. 「ORDER BY 店舗ｺｰﾄﾞ」を設定する。

3. 関連情報を取得・設定する（処理13：詳細仕様参照）。

4. 関連情報を取得・設定する（処理14：詳細仕様参照）。

5. 関連情報を取得・設定する（処理15：詳細仕様参照）。

6. 関連情報を取得・設定する（処理16：詳細仕様参照）。

7. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

7.1. 通常処理：関連情報を取得・設定する（処理17：詳細仕様参照）。

7.2. 通常処理：関連情報を取得・設定する（処理18：詳細仕様参照）。

7.3. 通常処理：「条件19を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

7.3.1. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

7.3.2. 条件成立時：「対象項目」を設定する。

7.3.3. 条件成立時：「対象項目」を設定する。

7.4. 通常処理：関連情報を取得・設定する（処理21：詳細仕様参照）。

7.5. 通常処理：繰返し条件「条件22を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

7.5.1. 繰返しの各回：店舗参照データを実行する。

7.6. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

7-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

7-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

7-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

7-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

7-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

7-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す
条件23を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

searchCustomers

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「searchCustomers」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理19：詳細仕様参照）。
2. 関連情報を取得・設定する（処理20：詳細仕様参照）。
3. 「C.生年月日 AS BIRTH_DATE, C.担当者コード AS STAFF_CODE FROM 」を設定する。
4. 「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4.1. 条件成立時：「 AND C.店舗コード = ?」を設定する。

1. 「条件21を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.1. 条件成立時：「 OR UPPER(C.カナ氏名) LIKE UPPER(?)」を設定する。

1. 「条件22を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

6.1. 条件成立時：「 AND TO_CHAR(C.電話番号) LIKE ?」を設定する。

1. 「 ORDER BY C.店舗コード, C.顧客番号, C.カナ氏名」を設定する。
2. 関連情報を取得・設定する（処理23：詳細仕様参照）。
3. 関連情報を取得・設定する（処理24：詳細仕様参照）。
4. 関連情報を取得・設定する（処理25：詳細仕様参照）。
5. 関連情報を取得・設定する（処理26：詳細仕様参照）。
6. 関連情報を取得・設定する（処理27：詳細仕様参照）。
7. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

13.1. 通常処理：関連情報を取得・設定する（処理28：詳細仕様参照）。

13.2. 通常処理：関連情報を取得・設定する（処理29：詳細仕様参照）。

13.3. 通常処理：関連情報を取得・設定する（処理30：詳細仕様参照）。

13.4. 通常処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

13.4.1. 条件成立時：「対象項目」を設定する。

13.5. 通常処理：「条件31を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

13.5.1. 条件成立時：関連情報を取得・設定する（処理32：詳細仕様参照）。

13.5.2. 条件成立時：「対象項目」を設定する。

13.5.3. 条件成立時：「対象項目」を設定する。

13.6. 通常処理：「条件33を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

13.6.1. 条件成立時：「対象項目」を設定する。

13.7. 通常処理：関連情報を取得・設定する（処理34：詳細仕様参照）。

13.8. 通常処理：繰返し条件「条件35を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

13.8.1. 繰返しの各回：関連情報を取得・設定する（処理36：詳細仕様参照）。

13.8.2. 繰返しの各回：顧客参照データを実行する。

13.9. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

13-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

13-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

13-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

13-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

13-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

13-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-E0FD738666D8

情報	内容
操作ID	SQL-E0FD738666D8
対象テーブル	店名テーブル
操作	参照

情報	内容
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	WHERE句による限定はこのSQL式にない。呼出条件・動的付加条件は原本を参照する。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。
原本	nenkin/WEB-INF/src/nenkin/service/ReferenceDataService.java:291 / loadBranchNames@289

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す
条件37を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

searchStaff

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「searchStaff」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理16：詳細仕様参照）。
2. 「S.所属店番 AS BRANCH_NO, S.役職コード AS ROLE_CODE FROM 」を設定する。

3. 「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：「 AND S.所属店番 = ?」を設定する。

1. 「条件17を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4.1. 条件成立時：「 OR UPPER(S.漢字氏名) LIKE UPPER(?)」を設定する。

1. 「 ORDER BY S.職員番号」を設定する。
2. 関連情報を取得・設定する（処理18：詳細仕様参照）。
3. 関連情報を取得・設定する（処理19：詳細仕様参照）。
4. 関連情報を取得・設定する（処理20：詳細仕様参照）。
5. 関連情報を取得・設定する（処理21：詳細仕様参照）。
6. 関連情報を取得・設定する（処理22：詳細仕様参照）。
7. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

11.1. 通常処理：関連情報を取得・設定する（処理23：詳細仕様参照）。

11.2. 通常処理：関連情報を取得・設定する（処理24：詳細仕様参照）。

11.3. 通常処理：関連情報を取得・設定する（処理25：詳細仕様参照）。

11.4. 通常処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

11.4.1. 条件成立時：「対象項目」を設定する。

11.5. 通常処理：「条件26を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

11.5.1. 条件成立時：関連情報を取得・設定する（処理27：詳細仕様参照）。

11.5.2. 条件成立時：「対象項目」を設定する。

11.5.3. 条件成立時：「対象項目」を設定する。

11.6. 通常処理：関連情報を取得・設定する（処理28：詳細仕様参照）。

11.7. 通常処理：繰返し条件「条件29を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

11.7.1. 繰返しの各回：関連情報を取得・設定する（処理30：詳細仕様参照）。

11.8. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

11-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

11-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

11-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

11-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

11-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

11-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-E0FD738666D8

情報	内容
操作ID	SQL-E0FD738666D8
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。

情報	内容
対象条件	WHERE句による限定はこのSQL式にない。呼出条件・動的付加条件は原本を参照する。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。
原本	nenkin/WEB-INF/src/nenkin/service/ReferenceDataService.java:291 / loadBranchNames@289

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す
条件31を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

getBranch

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「getBranch」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「WHERE 店舗ｺｰﾄﾞ = ?」を設定する。
2. 関連情報を取得・設定する（処理7：詳細仕様参照）。
3. 関連情報を取得・設定する（処理8：詳細仕様参照）。
4. 関連情報を取得・設定する（処理9：詳細仕様参照）。

5. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

5.1. 通常処理：関連情報を取得・設定する（処理10：詳細仕様参照）。

5.2. 通常処理：関連情報を取得・設定する（処理11：詳細仕様参照）。

5.3. 通常処理：「対象項目」を設定する。

5.4. 通常処理：関連情報を取得・設定する（処理12：詳細仕様参照）。

5.5. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

5-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

5-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

5-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

5-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

5-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

5-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-2CF7CA976867

情報	内容
操作ID	SQL-2CF7CA976867
対象テーブル	店名テーブル
操作	参照

情報	内容
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：店舗ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。
原本	nenkin/WEB-INF/src/nenkin/service/ReferenceDataService.java:185 / getBranch@184

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す
条件13を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

getStaff

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「getStaff」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「S WHERE S.職員番号 = ?」を設定する。
2. 関連情報を取得・設定する（処理8：詳細仕様参照）。
3. 関連情報を取得・設定する（処理9：詳細仕様参照）。

4. 関連情報を取得・設定する（処理10：詳細仕様参照）。

5. 関連情報を取得・設定する（処理11：詳細仕様参照）。

6. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

6.1. 通常処理：関連情報を取得・設定する（処理12：詳細仕様参照）。

6.2. 通常処理：関連情報を取得・設定する（処理13：詳細仕様参照）。

6.3. 通常処理：「対象項目」を設定する。

6.4. 通常処理：関連情報を取得・設定する（処理14：詳細仕様参照）。

6.5. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

6-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

6-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

6-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

6-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

6-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

6-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-D9F3500B923D

情報	内容
操作ID	SQL-D9F3500B923D

情報	内容
対象テーブル	DB2.U職員名簿表
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：S.職員番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。
原本	nenkin/WEB-INF/src/nenkin/service/ReferenceDataService.java:207 / getStaff@206

DB操作 SQL-E0FD738666D8

情報	内容
操作ID	SQL-E0FD738666D8
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	WHERE句による限定はこのSQL式にない。呼出条件・動的付加条件は原本を参照する。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。
原本	nenkin/WEB-INF/src/nenkin/service/ReferenceDataService.java:291 / loadBranchNames@289

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す
条件15を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

verifyStaff

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「verifyStaff」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-D9F3500B923D

情報	内容
操作ID	SQL-D9F3500B923D
対象テーブル	DB2.U職員名簿表

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：S.職員番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。
原本	nenkin/WEB-INF/src/nenkin/service/ReferenceDataService.java:207 / getStaff@206

DB操作 SQL-E0FD738666D8

情報	内容
操作ID	SQL-E0FD738666D8
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	WHERE句による限定はこのSQL式にない。呼出条件・動的付加条件は原本を参照する。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。
原本	nenkin/WEB-INF/src/nenkin/service/ReferenceDataService.java:291 / loadBranchNames@289

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

listGenders

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「listGenders」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

listClaimStatuses

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「listClaimStatuses」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

listBenefitStatuses

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「listBenefitStatuses」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getPensionTypeLabel

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「getPensionTypeLabel」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getPensionTypeLabels

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「getPensionTypeLabels」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理4：詳細仕様参照）。
2. 「年金種類」を設定する。
3. 繰返し条件「条件5を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

3.1. 繰返しの各回：関連情報を取得・設定する（処理6：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

listCodes

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「listCodes」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理11：詳細仕様参照）。
2. 関連情報を取得・設定する（処理12：詳細仕様参照）。
3. 関連情報を取得・設定する（処理13：詳細仕様参照）。
4. 関連情報を取得・設定する（処理14：詳細仕様参照）。
5. 関連情報を取得・設定する（処理15：詳細仕様参照）。
6. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

6.1. 通常処理：関連情報を取得・設定する（処理16：詳細仕様参照）。

6.2. 通常処理：関連情報を取得・設定する（処理17：詳細仕様参照）。

6.3. 通常処理：関連情報を取得・設定する（処理18：詳細仕様参照）。

6.4. 通常処理：繰返し条件「条件19を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

6.4.1. 繰返しの各回：関連情報を取得・設定する（処理20：詳細仕様参照）。

6.5. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

6-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

6-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

6-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

6-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

6-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

6-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す
条件21を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

loadBranchNames

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「loadBranchNames」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 「SELECT 店舗ｺｰﾄﾞ AS BRANCH_NO, 店名 AS BRANCH_NAME FROM 」を設定する。
3. 関連情報を取得・設定する（処理11：詳細仕様参照）。
4. 関連情報を取得・設定する（処理12：詳細仕様参照）。
5. 関連情報を取得・設定する（処理13：詳細仕様参照）。
6. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

6.1. 通常処理：関連情報を取得・設定する（処理14：詳細仕様参照）。

6.2. 通常処理：関連情報を取得・設定する（処理15：詳細仕様参照）。

6.3. 通常処理：関連情報を取得・設定する（処理16：詳細仕様参照）。

6.4. 通常処理：繰返し条件「条件17を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

6.4.1. 繰返しの各回：関連情報を取得・設定する（処理18：詳細仕様参照）。

6.5. 通常処理：終了時の共通後処理を行ってから、処理結果を呼出元へ返す。この経路の後続手順は実行しない。

6-例外1. 例外条件「SQLException e」に該当した場合に次の手順を行う。型名は原本の表記。

6-例外1.1. 例外発生時：終了時の共通後処理を行ってから、処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

6-後処理. 通常終了・例外・途中のreturnのいずれでも、この範囲を抜ける前に次の共通後処理を行う。

6-後処理.1. 終了時の共通後処理：使用した接続・処理資源を解放する。

6-後処理.2. 終了時の共通後処理：使用した接続・処理資源を解放する。

6-後処理.3. 終了時の共通後処理：使用した接続・処理資源を解放する。

DB操作の詳細

DB操作 SQL-E0FD738666D8

情報	内容
操作ID	SQL-E0FD738666D8
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	WHERE句による限定はこのSQL式にない。呼出条件・動的付加条件は原本を参照する。

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。
原本	nenkin/WEB-INF/src/nenkin/service/ReferenceDataService.java:291 / loadBranchNames@289

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す
条件19を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する

readStaff

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「readStaff」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理3：詳細仕様参照）。
2. 関連情報を取得・設定する（処理4：詳細仕様参照）。
3. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

nullableInt

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「nullableInt」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

displayPhone

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「displayPhone」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理3：詳細仕様参照）。
2. 「(value.length() が等しい 9 || value.length() が等しい 10)、かつ条件4を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。
   該当しない場合は分岐の後へ進む。

2.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
(value.length() が等しい 9 || value.length() が等しい 10)、かつ条件5を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT ((value.length() が等しい 9 || value.length() が等しい 10) && !value.startsWith("0"))	処理結果を呼出元へ返す

compactDate

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「compactDate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 「value.length() が等しい 0、または条件7を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理8：詳細仕様参照）。
2. 「条件9を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4.1. 条件成立時：関連情報を取得・設定する（処理10：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
value.length() が等しい 0、または条件11を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT (value.length() が等しい 0 || "0".equals(value))	処理結果を呼出元へ返す

canonicalPhone

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「canonicalPhone」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理5：詳細仕様参照）。
2. 関連情報を取得・設定する（処理6：詳細仕様参照）。
3. 繰返し条件「条件7を満たす（原本条件は補足説明を参照）、かつdigits.charAt(first) が等しい '0'」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

3.1. 繰返しの各回：関連情報を取得・設定する（処理8：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

normalize

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「normalize」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

text

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「text」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

containsPattern

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「containsPattern」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

failure

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「failure」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

close

処理概要

Access と同じ店舗・職員・顧客・コード表を参照する JDBC サービスで「close」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：使用した接続・処理資源を解放する。

2-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

結果・例外時の扱い

到達条件	結果
対象項目がない	処理結果を呼出元へ返す

DB関連処理

補足説明

適用条件

各仮定のQA番号を確認する。静的に確認できた操作・条件を適用する。

処理・条件の原本対応

文書内の条件番号・処理番号は次の原本表記に対応する。完全な業務用語への置換を行っていない箇所はQA・reportで確認する。

イベント	参照番号	原本条件・処理
searchBranches	条件1	results.next()
searchBranches	処理2	results = statement.executeQuery()
searchBranches	処理3	String pattern = containsPattern(value)
searchBranches	条件4	value.length() > 0
searchBranches	処理5	statement = connection.prepareStatement(sql)
searchBranches	処理6	connection = NenkinDataSource.openConnection()
searchBranches	処理7	ResultSet results = null
searchBranches	処理8	PreparedStatement statement = null
searchBranches	処理9	Connection connection = null

イベント	参照番号	原本条件・処理
searchBranches	処理10	List<BranchLookupItem> rows = new ArrayList<BranchLookupItem>()
searchBranches	処理11	String value = normalize(keyword)
searchBranches	処理12	String value = normalize(keyword)
searchBranches	処理13	List<BranchLookupItem> rows = new ArrayList<BranchLookupItem>()
searchBranches	処理14	Connection connection = null
searchBranches	処理15	PreparedStatement statement = null
searchBranches	処理16	ResultSet results = null
searchBranches	処理17	connection = NenkinDataSource.openConnection()
searchBranches	処理18	statement = connection.prepareStatement(sql)
searchBranches	条件19	value.length() > 0
searchBranches	処理20	String pattern = containsPattern(value)
searchBranches	処理21	results = statement.executeQuery()
searchBranches	条件22	results.next()
searchBranches	条件23	SQLException e
searchCustomers	条件1	results.next()
searchCustomers	処理2	int rowBranch = results.getInt("BRANCH_NO")
searchCustomers	処理3	results = statement.executeQuery()
searchCustomers	条件4	phone.length() > 0

イベント	参照番号	原本条件・処理
searchCustomers	処理5	String pattern = containsPattern(name)
searchCustomers	条件6	name.length() > 0
searchCustomers	処理7	int index = 1
searchCustomers	処理8	statement = connection.prepareStatement(sql.toString())
searchCustomers	処理9	connection = NenkinDataSource.openCustomerConnection()
searchCustomers	処理10	ResultSet results = null
searchCustomers	処理11	PreparedStatement statement = null
searchCustomers	処理12	Connection connection = null
searchCustomers	処理13	Map<Integer, String> branchNames = loadBranchNames()
searchCustomers	処理14	List<CustomerLookupItem> rows = new ArrayList<CustomerLookupItem>()
searchCustomers	条件15	phone.length() > 0
searchCustomers	条件16	name.length() > 0
searchCustomers	処理17	String phone = canonicalPhone(phoneKeyword)
searchCustomers	処理18	String name = normalize(nameKeyword)
searchCustomers	処理19	String name = normalize(nameKeyword)
searchCustomers	処理20	String phone = canonicalPhone(phoneKeyword)
searchCustomers	条件21	name.length() > 0
searchCustomers	条件22	phone.length() > 0

イベント	参照番号	原本条件・処理
searchCustomers	処理23	List<CustomerLookupItem> rows = new ArrayList<CustomerLookupItem>()
searchCustomers	処理24	Map<Integer, String> branchNames = loadBranchNames()
searchCustomers	処理25	Connection connection = null
searchCustomers	処理26	PreparedStatement statement = null
searchCustomers	処理27	ResultSet results = null
searchCustomers	処理28	connection = NenkinDataSource.openCustomerConnection()
searchCustomers	処理29	statement = connection.prepareStatement(sql.toString())
searchCustomers	処理30	int index = 1
searchCustomers	条件31	name.length() > 0
searchCustomers	処理32	String pattern = containsPattern(name)
searchCustomers	条件33	phone.length() > 0
searchCustomers	処理34	results = statement.executeQuery()
searchCustomers	条件35	results.next()
searchCustomers	処理36	int rowBranch = results.getInt("BRANCH_NO")
searchCustomers	条件37	SQLException e
searchStaff	条件1	results.next()
searchStaff	処理2	rows.add(readStaff(results, branchNames))
searchStaff	処理3	results = statement.executeQuery()
searchStaff	処理4	String pattern = containsPattern(keyword)

イベント	参照番号	原本条件・処理
searchStaff	条件5	keyword.length() > 0
searchStaff	処理6	int index = 1
searchStaff	処理7	statement = connection.prepareStatement(sql.toString())
searchStaff	処理8	connection = NenkinDataSource.openStaffConnection()
searchStaff	処理9	ResultSet results = null
searchStaff	処理10	PreparedStatement statement = null
searchStaff	処理11	Connection connection = null
searchStaff	処理12	Map<Integer, String> branchNames = loadBranchNames()
searchStaff	処理13	List<StaffLookupItem> rows = new ArrayList<StaffLookupItem>()
searchStaff	条件14	keyword.length() > 0
searchStaff	処理15	String keyword = normalize(kanaKeyword)
searchStaff	処理16	String keyword = normalize(kanaKeyword)
searchStaff	条件17	keyword.length() > 0
searchStaff	処理18	List<StaffLookupItem> rows = new ArrayList<StaffLookupItem>()
searchStaff	処理19	Map<Integer, String> branchNames = loadBranchNames()
searchStaff	処理20	Connection connection = null
searchStaff	処理21	PreparedStatement statement = null
searchStaff	処理22	ResultSet results = null
searchStaff	処理23	connection = NenkinDataSource.openStaffConnection()

イベント	参照番号	原本条件・処理
searchStaff	処理24	statement = connection.prepareStatement(sql.toString())
searchStaff	処理25	int index = 1
searchStaff	条件26	keyword.length() > 0
searchStaff	処理27	String pattern = containsPattern(keyword)
searchStaff	処理28	results = statement.executeQuery()
searchStaff	条件29	results.next()
searchStaff	処理30	rows.add(readStaff(results, branchNames))
searchStaff	条件31	SQLException e
getBranch	処理1	results = statement.executeQuery()
getBranch	処理2	statement = connection.prepareStatement(sql)
getBranch	処理3	connection = NenkinDataSource.openConnection()
getBranch	処理4	ResultSet results = null
getBranch	処理5	PreparedStatement statement = null
getBranch	処理6	Connection connection = null
getBranch	処理7	Connection connection = null
getBranch	処理8	PreparedStatement statement = null
getBranch	処理9	ResultSet results = null
getBranch	処理10	connection = NenkinDataSource.openConnection()
getBranch	処理11	statement = connection.prepareStatement(sql)

イベント	参照番号	原本条件・処理
getBranch	処理12	results = statement.executeQuery()
getBranch	条件13	SQLException e
getStaff	処理1	results = statement.executeQuery()
getStaff	処理2	statement = connection.prepareStatement(sql)
getStaff	処理3	connection = NenkinDataSource.openStaffConnection()
getStaff	処理4	ResultSet results = null
getStaff	処理5	PreparedStatement statement = null
getStaff	処理6	Connection connection = null
getStaff	処理7	Map<Integer, String> branchNames = loadBranchNames()
getStaff	処理8	Map<Integer, String> branchNames = loadBranchNames()
getStaff	処理9	Connection connection = null
getStaff	処理10	PreparedStatement statement = null
getStaff	処理11	ResultSet results = null
getStaff	処理12	connection = NenkinDataSource.openStaffConnection()
getStaff	処理13	statement = connection.prepareStatement(sql)
getStaff	処理14	results = statement.executeQuery()
getStaff	条件15	SQLException e
verifyStaff	処理1	StaffLookupItem staff = getStaff(staffNo)
verifyStaff	処理2	StaffLookupItem staff = getStaff(staffNo)

イベント	参照番号	原本条件・処理
getPensionTypeLabel	処理1	String label = getPensionTypeLabels().get(Integer.valueOf(code))
getPensionTypeLabel	処理2	String label = getPensionTypeLabels().get(Integer.valueOf(code))
getPensionTypeLabels	条件1	int i = 0; i < rows.size(); i++
getPensionTypeLabels	処理2	labels.put(Integer.valueOf(rows.get(i).getCode()), rows.get(i).getLabel())
getPensionTypeLabels	処理3	Map<Integer, String> labels = new HashMap<Integer, String>()
getPensionTypeLabels	処理4	Map<Integer, String> labels = new HashMap<Integer, String>()
getPensionTypeLabels	条件5	int i = 0; i < rows.size(); i++
getPensionTypeLabels	処理6	labels.put(Integer.valueOf(rows.get(i).getCode()), rows.get(i).getLabel())
listCodes	条件1	results.next()
listCodes	処理2	rows.add(new CodeLookupItem(results.getInt("CODE_VALUE"), results.getString("LABEL_VALUE")))
listCodes	処理3	results = statement.executeQuery()
listCodes	処理4	statement = connection.prepareStatement(sql)
listCodes	処理5	connection = NenkinDataSource.openConnection()
listCodes	処理6	ResultSet results = null
listCodes	処理7	PreparedStatement statement = null
listCodes	処理8	Connection connection = null
listCodes	処理9	List<CodeLookupItem> rows = new ArrayList<CodeLookupItem>()

イベント	参照番号	原本条件・処理
listCodes	処理10	String sql = "SELECT " + codeColumn + " AS CODE_VALUE, " + labelColumn + " AS LABEL_VALUE FROM " + table + " ORDER BY " + codeColumn
listCodes	処理11	String sql = "SELECT " + codeColumn + " AS CODE_VALUE, " + labelColumn + " AS LABEL_VALUE FROM " + table + " ORDER BY " + codeColumn
listCodes	処理12	List<CodeLookupItem> rows = new ArrayList<CodeLookupItem>()
listCodes	処理13	Connection connection = null
listCodes	処理14	PreparedStatement statement = null
listCodes	処理15	ResultSet results = null
listCodes	処理16	connection = NenkinDataSource.openConnection()
listCodes	処理17	statement = connection.prepareStatement(sql)
listCodes	処理18	results = statement.executeQuery()
listCodes	条件19	results.next()
listCodes	処理20	rows.add(new CodeLookupItem(results.getInt("CODE_VALUE"), results.getString("LABEL_VALUE")))
listCodes	条件21	SQLException e
loadBranchNames	条件1	results.next()
loadBranchNames	処理2	branchNames.put(Integer.valueOf(results.getInt("BRANCH_NO")), results.getString("BRANCH_NAME"))
loadBranchNames	処理3	results = statement.executeQuery()
loadBranchNames	処理4	statement = connection.prepareStatement(sql)

イベント	参照番号	原本条件・処理
loadBranchNames	処理5	connection = NenkinDataSource.openConnection()
loadBranchNames	処理6	ResultSet results = null
loadBranchNames	処理7	PreparedStatement statement = null
loadBranchNames	処理8	Connection connection = null
loadBranchNames	処理9	Map<Integer, String> branchNames = new HashMap<Integer, String>()
loadBranchNames	処理10	Map<Integer, String> branchNames = new HashMap<Integer, String>()
loadBranchNames	処理11	Connection connection = null
loadBranchNames	処理12	PreparedStatement statement = null
loadBranchNames	処理13	ResultSet results = null
loadBranchNames	処理14	connection = NenkinDataSource.openConnection()
loadBranchNames	処理15	statement = connection.prepareStatement(sql)
loadBranchNames	処理16	results = statement.executeQuery()
loadBranchNames	条件17	results.next()
loadBranchNames	処理18	branchNames.put(Integer.valueOf(results.getInt("BRANCH_NO")), results.getString("BRANCH_NAME"))
loadBranchNames	条件19	SQLException e
readStaff	処理1	String branchName = branchNames.get(Integer.valueOf(branchNo))
readStaff	処理2	int branchNo = results.getInt("BRANCH_NO")

イベント	参照番号	原本条件・処理
readStaff	処理3	int branchNo = results.getInt("BRANCH_NO")
readStaff	処理4	String branchName = branchNames.get(Integer.valueOf(branchNo))
nullableInt	処理1	int value = results.getInt(column)
nullableInt	処理2	int value = results.getInt(column)
displayPhone	条件1	!value.startsWith("0")
displayPhone	処理2	String value = canonicalPhone(rawValue)
displayPhone	処理3	String value = canonicalPhone(rawValue)
displayPhone	条件4	!value.startsWith("0")
displayPhone	条件5	!value.startsWith("0")
compactDate	処理1	value = value.substring(0, decimal)
compactDate	条件2	decimal >= 0
compactDate	処理3	int decimal = value.indexOf('.')
compactDate	条件4	"0".equals(value)
compactDate	処理5	String value = text(rawValue).trim()
compactDate	処理6	String value = text(rawValue).trim()
compactDate	条件7	"0".equals(value)
compactDate	処理8	int decimal = value.indexOf('.')
compactDate	条件9	decimal >= 0
compactDate	処理10	value = value.substring(0, decimal)

イベント	参照番号	原本条件・処理
compactDate	条件11	"0".equals(value)
canonicalPhone	条件1	first < digits.length() - 1
canonicalPhone	処理2	first++
canonicalPhone	処理3	int first = 0
canonicalPhone	処理4	String digits = PensionRepository.digits(value)
canonicalPhone	処理5	String digits = PensionRepository.digits(value)
canonicalPhone	処理6	int first = 0
canonicalPhone	条件7	first < digits.length() - 1
canonicalPhone	処理8	first++
containsPattern	処理1	String escaped = text(value).replace("\", "\\") .replace("%", "\%").replace("", "\") .replace("[", "\[").replace("]", "\]")
containsPattern	処理2	String escaped = text(value).replace("\", "\\") .replace("%", "\%").replace("", "\") .replace("[", "\[").replace("]", "\]")